

Accord — Project Thesis

Status

Draft v0.1

Working Title

Accord

Tagline

Scoped context. Explicit approval. Portable trust.

1. One-Sentence Thesis

Accord is an open protocol and developer SDK for **scoped context disclosure** and **human approval of high-impact AI actions**, designed to make personal AI systems more portable, inspectable, and trustworthy.

2. The Core Problem

AI systems are rapidly becoming more personal and more capable. They can connect to tools, remember preferences, and take actions on behalf of users. But there is still no clean, developer-friendly standard for answering a few basic questions:

- What user context is the AI using right now?
- What scope of access was granted, and for how long?
- Which actions should require explicit human approval?
- How can a user inspect, revoke, or export those decisions later?
- How can this behavior remain portable across products instead of being trapped inside one assistant?

Today, these concerns are handled inconsistently across products. Some systems remember too much, some disclose too little, and some act without a sufficiently clear approval boundary. Existing tooling covers adjacent pieces of the problem, but the control plane for **how personal context is disclosed and how risky actions are approved** remains fragmented.

3. Why This Project Should Exist

The next wave of AI products will not win purely on raw model quality. They will win on **dependability**.

For users to trust personal AI, they need more than memory and tool access. They need:

- **Visibility** into what context is being used
- **Control** over what can be used and when
- **Approval** before high-impact actions occur
- **Auditability** after the fact
- **Portability** across applications

For developers, these needs are currently painful to implement well. Every product ends up inventing its own partial solution for context access, consent prompts, approval flows, and logging. That is inefficient, inconsistent, and difficult to standardize.

Accord exists to provide the missing layer between:

- tool/data connectivity,
 - personal context and memory,
 - and real-world action execution.
-

4. What Accord Is

Accord is:

- an **open protocol** for disclosing personal context use and capturing action approval,
- a **TypeScript-first SDK** for developers building AI products,
- a **local-first reference implementation** for storage, policies, and audit records,
- and a **minimal approval console** where users can inspect context usage and approve or reject actions.

Accord is intended to complement existing ecosystems rather than replace them.

5. What Accord Is Not

Accord is **not**:

- a general AI assistant,
- a chatbot product,
- a hosted consumer SaaS in v1,
- a replacement for tool protocols,
- a replacement for memory engines,
- a broad “AI operating system,”

- or a full identity/compliance platform.

The project should stay narrow and opinionated around one core problem: **approval-aware personalization for AI systems**.

6. Project Positioning

Accord sits in the gap between three existing layers:

1. **Tool connectivity** — how AI reaches tools and external systems
2. **Memory/context systems** — how AI stores and retrieves user information
3. **Action execution** — how AI actually performs writes, sends, updates, or other side effects

Accord focuses on the missing control layer:

- what context is disclosed,
- what scope is permitted,
- when approval is required,
- and how the decision is recorded.

In short:

MCP can connect an AI to tools. Memory systems can persist user context. Accord decides how context is disclosed and when action execution must stop for human approval.

7. Who This Is For

Primary Users

Developers building AI applications, agents, copilots, and personal workflow tools who want a clean, reusable way to add:

- scoped context access,
- approval gates,
- audit trails,
- and revocation behavior.

Secondary Users

AI power users and technically inclined end users who want:

- clearer visibility into what an AI knows,
- control over what it can use,
- approval before risky actions,

- and portable trust across products.
-

8. The User Promise

For developers:

“Add safe, inspectable personalization and approval-aware actions to your AI app without reinventing the control layer.”

For end users:

“Your AI can remember what matters, show what it used, and ask before acting.”

9. Design Principles

1. Explicit over implicit

High-impact actions must not hide behind vague assistant behavior. If an AI wants to perform a meaningful external action, it should generate a structured proposal and disclose the relevant context it used.

2. Scoped access by default

Access to user context should be bounded by category, purpose, and time window whenever possible.

3. Local-first in v1

The first version should avoid unnecessary cloud dependence. Users and developers should be able to inspect the full system locally.

4. Portable by design

Context and approval records should not be trapped inside one application. Export, revoke, and adapter-friendly integration should be part of the design from the start.

5. Minimal but rigorous

The first version should solve one problem well rather than pretending to solve the full future of personal AI.

6. Human approval is a product primitive

Approval is not a patch added at the end. It is a first-class part of the runtime flow.

10. V1 Scope

Accord v1 should focus on **scoped context disclosure** and **approval-required action execution**.

Included in v1

- Protocol objects for context disclosure and approvals
- TypeScript SDK
- Local reference server
- Basic policy engine
- Minimal approval and audit console
- One adapter path to demonstrate real integration
- Two sample apps that show the protocol in action

Excluded from v1

- Deep third-party consumer integrations
 - Autonomous commerce flows
 - Broad memory ranking/retrieval infrastructure
 - Hosted multi-tenant productization
 - Mobile-first experience
 - Enterprise compliance features
 - Full agent orchestration framework
-

11. Example Use Cases

Use Case A: Coding Agent

A coding agent wants to edit files and open a commit using stored project preferences. Before writing or committing, it discloses which preferences it used and requests approval.

Use Case B: Research-to-Publish Agent

A writing agent uses a user's style preferences and publishing rules to draft content. Before publishing or opening a pull request, it generates an action proposal for review.

Use Case C: Personal Workflow Agent (Future)

A personal workflow agent wants to send a message, submit a form, or schedule an action based on user context. The system checks scope and requires approval before execution.

12. Why This Could Matter

If AI becomes more capable while remaining opaque, users will distrust it. If it becomes more personal without being portable, every assistant becomes a silo. If it becomes more agentic without approval boundaries, it becomes harder to safely adopt.

Accord matters because it treats **trust as infrastructure**.

It is a bet that the next important layer in AI is not just better models or more tools, but better control over how context and action intersect.

13. Success Criteria

Accord will be on the right track if:

- developers can integrate it into a prototype in hours, not weeks,
 - action proposals are simple and clear enough for real approval workflows,
 - users can inspect what context was used for a given action,
 - the boundary between allowed context use and approval-required action is easy to understand,
 - the protocol feels composable rather than framework-locked,
 - and early adopters see it as a missing primitive rather than another optional wrapper.
-

14. Main Risks

Risk 1: Too theoretical

If the project reads like a standards exercise without obvious developer value, adoption will be weak.

Mitigation: ship SDK, local server, console, and sample apps early.

Risk 2: Too broad

If the scope expands into memory systems, identity, orchestration, and privacy infrastructure all at once, the project will lose clarity.

Mitigation: keep v1 centered on disclosure, approval, and audit.

Risk 3: Too narrow

If the project is only a nicer confirmation dialog, it will not be meaningful infrastructure.

Mitigation: make the protocol expressive enough to carry scope, provenance, policy decisions, receipts, and revocation history.

Risk 4: Mispositioned against existing standards

If Accord is framed as competing with adjacent standards and ecosystems, it may face unnecessary resistance.

Mitigation: explicitly position it as complementary to tool connectivity and memory systems.

15. Initial Strategic Bet

The strongest path is not to build another assistant. It is to build the missing control layer that other assistants can adopt.

That means Accord should begin as:

- **protocol first,**
 - **SDK second,**
 - **reference implementation third,**
 - and **sample apps as proof, not the product itself.**
-

16. Open Questions

These questions should be answered in the next documents:

1. What exact protocol objects belong in v0.1?
 2. Which actions are considered high-impact in the policy model?
 3. How should context categories and sensitivity levels be represented?
 4. What should the minimal developer integration look like?
 5. How should Accord align with adjacent standards without becoming dependent on them?
 6. What is the smallest sample app set that proves the concept clearly?
-

17. Draft Closing Statement

Accord is an open protocol and developer SDK for a future where AI is personal but not opaque, capable but not unchecked, and portable without becoming ungovernable.

It exists to make AI systems more trustworthy by making context use explicit and action approval unavoidable where it matters.